

```
Study/sem2/ProjectElective/Final Test# ./med
Medium Priority Process arrived at : 1522220813
Press any button to enter into critical section
```

Medium Priority Process

Lo continues its task and completes it in the desired time. After this, *Hi* acquires the lock and performs its task.

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./low
My pid: 677
Press any button to enter into critical section
```

```
Aquired lock...
futex_add : 677
Low Priority Process started working at : 1522220815
Priority : 20
Priority : 20
Work completed at : 1522220830
Low Priority Process releasing lock...futex_add : 0
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test#
```

Low Priority Process

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./high
High Priority Process arrived at 1522220809 and waiting for
lock
My pid: 679
Press any button to enter into critical section
```

```
futex_add : 677
Waiting for lock...
High Priority Process started working at : 1522220830
Priority : 38
Priority : 38
Work completed at : 1522220835
High Priority Process releasing lock...
futex_add : 0
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume /
Study/sem2/P ojectElective/Final Test#
```

High Priority Process

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./med
Medium Priority Process arrived at : 1522220813
Press any button to enter into critical section
```

```
Medium Priority Process started working at : 1522220835
Priority : 28
Priority : 28
```

```
Work completed at : 1522220840
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test#
```

Medium Priority Process

After the completion of *Hi*, *Med* is allowed to start its work. Here, even though all processes are executing on a single-core processor, there is no extra time delay in order to provide resources to the high priority process (*Hi*).

Multi-core operation: All the code for *sem.c*, *Low.c*, *High.c* and *Medium.c* is compiled and the executables are stored as *a.out*, *low*, *high* and *med*, respectively. *a.out* is executed first, and then all the remaining run almost together.

Now, instead of running all processes on a single core, if we allow processes to execute their tasks on different cores, then we can execute processes in parallel. Hence, the priority inversion problem will not occur. Here, different CPU cores are allocated to *Hi*, *Lo* and *Med* by setting different CPU affinities. We have observed that when *Lo* has a semaphore and performs the CS task, *Hi* is waiting for semaphore and *Med* arrives, which then starts performing its own task on another core in a parallel manner. Hence, no extra delay is observed in the execution time of *Lo*, and *Hi* gets resources as soon as *Lo* completes execution and starts its work, even though *Med* is performing its task on another core.

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./low
Low process arrived at : 1522160686
Press any button to enter into critical section
```

```
start Time: 1522160699
Low Priority Process started working at : 1522160699
My Priority is : 20
```

Low Priority Process

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./high
High process arrived at : 1522160693
Press any button to enter into critical section
```

High process waiting for lock

High Priority Process

```
root@dhanashree-Inspiron-5559:/media/dhanashree/New Volume/
Study/sem2/ProjectElective/Final Test# ./med
Med Priority Process arrived at: 1522160696
Press any button to start work...
```

```
start Time: 1522160702
```